

SONARQUBE STATIC CODE ANALYSIS REPORT

Submitted By: Rishabh Gupta

Project Name: Address Book Application

Technology Stack: Java, Maven, JUnit 5, JaCoCo, SonarQube

1. Introduction to SonarQube

SonarQube is an open-source platform used for continuous inspection of code quality. It performs static code analysis to detect bugs, vulnerabilities, code smells, and maintainability issues in software projects. SonarQube helps developers improve code quality by providing detailed metrics, reports, and quality gates that ensure coding standards are maintained. It supports multiple programming languages and integrates seamlessly with build tools such as Maven and Gradle. In this project, SonarQube is integrated with a Java Maven application to analyze code quality and measure test coverage.

2. Objectives of the Assignment

- 1 Integrate SonarQube with a Java Maven project.
- 2 Perform static code analysis on the Address Book Application.
- 3 Measure unit testing coverage using JaCoCo.
- 4 Evaluate reliability, security, and maintainability of the project.
- 5 Generate a professional code quality report for submission.

3. Project Overview – Address Book Application

The Address Book Application is a Java-based contact management system that allows users to store, update, retrieve, and delete contact information. The project follows a layered architecture to ensure separation of concerns and maintainability of the code. The system includes model classes, service layer logic, and data access components. Unit testing is implemented using JUnit, and code coverage is measured using JaCoCo.

- 1 Model Layer – Defines contact data structure.
- 2 DAO Layer – Handles storage and retrieval operations.
- 3 Service Layer – Implements business logic and validations.
- 4 Main Layer – Application entry point and user interaction.

4. SonarQube Setup and Integration

- 1 Installed SonarQube Community Edition on local machine.
- 2 Started SonarQube server and accessed dashboard using localhost.
- 3 Generated authentication token from SonarQube.
- 4 Configured Maven project with SonarQube plugin.
- 5 Executed Maven command to run analysis and upload results.
- 6 Integrated JaCoCo plugin for code coverage reporting.

5. SonarQube Analysis Results

- 1 Quality Gate Status: PASSED

- 2 Reliability Rating: A
- 3 Security Rating: A
- 4 Maintainability Rating: A
- 5 Number of Bugs: 0
- 6 Number of Vulnerabilities: 0
- 7 Code Smells: Minor and manageable
- 8 Code Duplication: 0%
- 9 Test Coverage measured using JaCoCo

6. JaCoCo Code Coverage

JaCoCo was integrated with the Maven build lifecycle to generate detailed code coverage metrics. Coverage reports include line coverage, branch coverage, method coverage, and class coverage. Reports are generated in HTML format and stored in the project directory: `target/site/jacoco/index.html`

7. Benefits of Using SonarQube

- 1 Improves overall code quality and maintainability.
- 2 Detects bugs and security vulnerabilities early.
- 3 Provides automated quality gates.
- 4 Encourages best coding practices.
- 5 Helps teams maintain clean and reliable code.

8. Conclusion

The integration of SonarQube with the Address Book Application was successfully completed. Static code analysis confirmed high code quality with no critical issues. The project passed the quality gate and demonstrated strong reliability and maintainability. JaCoCo coverage provided insights into test effectiveness. Overall, SonarQube proved to be an effective tool for monitoring and improving software quality in a professional development environment.